Tauler de Catan: com llegir els fitxers de topologia

M0486 Accés a dades · BA2 Hibernate · Institut Sabadell

El tauler de Catan no s'escriu al codi: es llegeix de quatre fitxers CSV que hi ha a `src/main/resources/tauler/`. Aquests fitxers només descriuen la **forma** del tauler (quins hexàgons hi ha, on són els vèrtexs i quines arestes els uneixen). El terreny i el número de cada hexàgon **no hi són**: es sortegen a IniciarPartida.

Format comú: UTF-8, separador coma, la primera línia és la capçalera i el decimal és el punt (`-0.866`).

Fitxer	Capçalera	Files	Què representa	Classe Java
`hexagons.csv`	`codi,q,r`	19	Cada casella del tauler i la seva posició a la graella	Hexagon
`vertexs.csv`	`codi,x,y`	54	Cada cantonada on es pot construir un poblat o una ciutat	Vertex
`hexagon_vertex.csv`	`hexagon,vertex,posicio`	114	Quins 6 vèrtexs té cada hexàgon i en quin ordre	HexagonVertex
`arestes.csv`	`codi,vertexA,vertexB`	72	Cada costat on es pot construir una carretera	Aresta

El `codi` no és l'`id`. Cada partida crea el seu propi tauler, per tant a la BD hi haurà 54 vèrtexs per partida. A la primera partida l'id i el codi poden coincidir, però a la segona ja no. El `codi` del CSV es guarda al camp `codi` de l'entitat i serveix per enllaçar els fitxers durant la càrrega; els mètodes del DAO i els endpoints treballen sempre amb l'`id` de la BD.

Ordre de càrrega

Una fila no pot apuntar a una entitat que encara no existeix. Per això:

1. `vertexs.csv` → es creen els 54 Vertex i es guarden en un `Map<Integer, Vertex>` pel codi.
2. `hexagons.csv` → es creen els 19 Hexagon (amb el terreny i el número sortejats).
3. `hexagon_vertex.csv` → per cada fila es busca l'hexàgon i el vèrtex als mapes i es crea el HexagonVertex.
4. `arestes.csv` → per cada fila es busquen els dos vèrtexs al mapa i es crea l'Aresta.

1. El sistema de coordenades

Tot el tauler es dibuixa amb hexàgons **de punta amunt** (un vèrtex a dalt i un a baix). Les mesures estan en una unitat abstracta: **1 unitat = distància del centre d'un hexàgon a qualsevol dels seus vèrtexs**. Per tant, un hexàgon fa 2 unitats d'alt i $\sqrt{3} \approx 1{,}732$ d'ample.

Els eixos segueixen el criteri de la pantalla (Swing, JavaFX, canvas d'HTML): la **x creix cap a la dreta** i la **y creix cap avall**. El punt (0, 0) és el centre del tauler, que coincideix amb el centre de l'hexàgon 10. Totes les coordenades van de $-4{,}33$ a $4{,}33$ en x i de -4 a 4 en y.

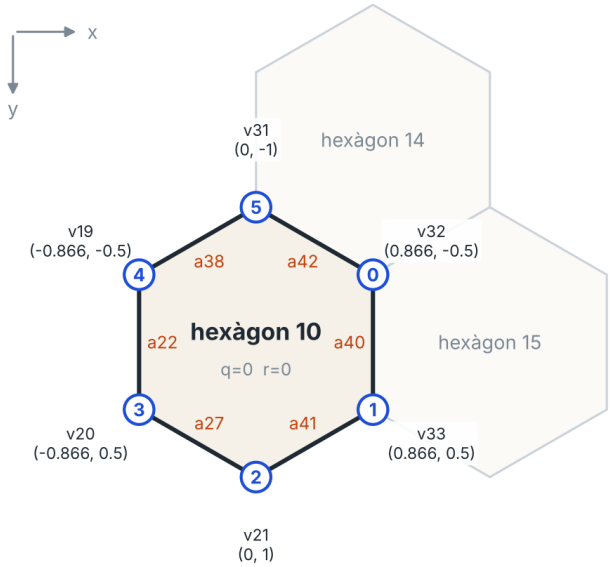


Figura 1. L'hexàgon central (codi 10). En blau, la posició de cada vèrtex dins l'hexàgon (0-5) i, al costat, el seu codi i les seves coordenades. En taronja, el codi de les 6 arestes.

La `posicio` 0 és sempre el vèrtex de **dalt a la dreta** i es continua en **sentit horari**: 1 baix a la dreta, 2 a baix, 3 baix a l'esquerra, 4 dalt a l'esquerra i 5 a dalt. Això és el mateix per als 19 hexàgons, de manera que si s'agafen els vèrtexs d'un hexàgon ordenats per posició i s'uneixen en aquest ordre, surt l'hexàgon dibuixat.

2. hexagons.csv

Columna	Tipus	Significat
`codi`	int	Identificador de l'hexàgon dins dels CSV (1-19). Es guarda a Hexagon.codi.
`q`	int	Columna a la graella hexagonal (de −2 a 2). Creix cap a la dreta.
`r`	int	Fila a la graella hexagonal (de −2 a 2). Creix cap avall.

`q` i `r` són **coordenades axials**: les habituals en graelles hexagonals. Una fila del tauler té la mateixa `r`; en canvi, la `q` constant segueix una diagonal. Un hexàgon és dins del tauler si $|q| \le 2$, $|r| \le 2$ i $|q + r| \le 2$, i per això n'hi ha 19. El centre de l'hexàgon es calcula així:

```
centreX = √3 · (q + r / 2)        centreY = 1,5 · r
Exemple: hexàgon 15 (q=1, r=0)  →  (1,732 ; 0)
```

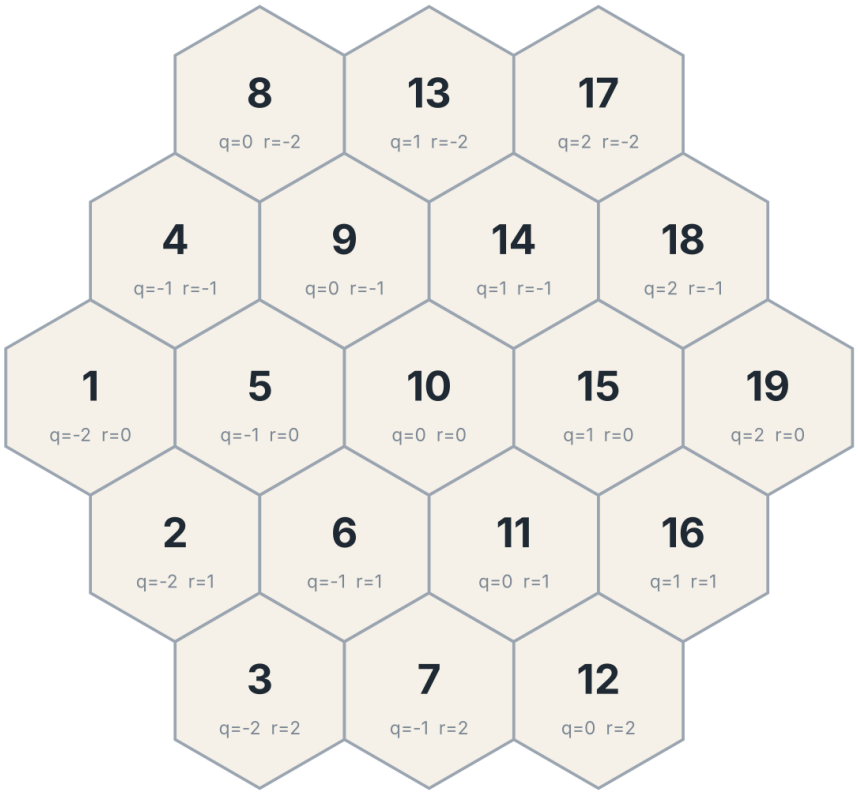


Figura 2. Codi de cada hexàgon i les seves coordenades (q, r).

Fixeu-vos que els codis no van per files: el CSV recorre les columnes q = −2, −1, 0, 1, 2 i, dins de cada columna, la r de petita a gran. Per això l'hexàgon 1 és a l'esquerra de la fila central i no a dalt de tot. Per a la pràctica no importa, perquè el codi només serveix per enllaçar fitxers.

3. vertexs.csv

Columna	Tipus	Significat
`codi`	int	Identificador del vèrtex dins dels CSV (1-54). Es guarda a Vertex.codi.
x	double	Coordenada horitzontal, en unitats (radi de l'hexàgon = 1). Arrodonida a 3 decimals.
y	double	Coordenada vertical, en unitats, cap avall.

Els valors de x i y hi són tots (no hi ha cap cel·la buida). Un mateix vèrtex el comparteixen fins a 3 hexàgons, però al fitxer surt **una sola vegada**: és el mateix punt del tauler i, per tant, el mateix lloc on hi pot haver un únic poblat.

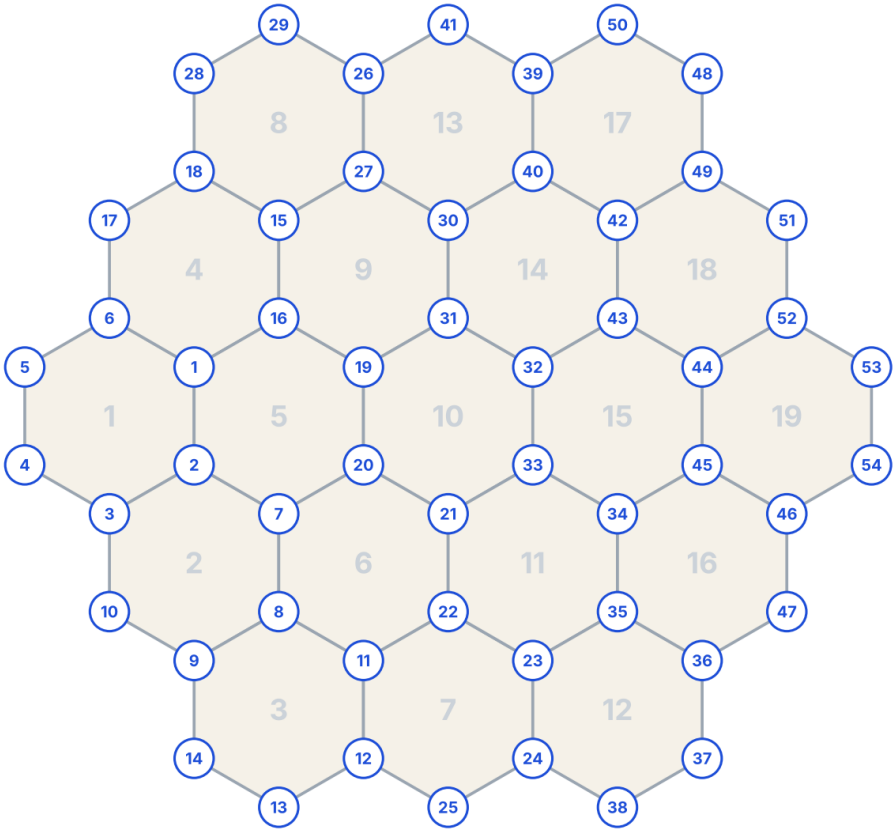


Figura 3. Codi de cada vèrtex. En gris clar, el codi de l'hexàgon.

Vèrtexs que toquen…	Quants	On són
1 hexàgon	18	A la costa, a les puntes exteriors
2 hexàgons	12	A la costa, entre dos hexàgons
3 hexàgons	24	A l'interior del tauler

4. hexagon_vertex.csv

Columna	Tipus	Significat
`hexagon`	int	Codi de l'hexàgon (hexagons.csv).
`vertex`	int	Codi del vèrtex (vertexs.csv).
`posicio`	int	Lloc del vèrtex dins d'aquest hexàgon (0-5, figura 1).

És la relació N-M entre hexàgons i vèrtexs. Com que la relació té un atribut propi (la posició), no pot ser un `@ManyToMany`: és l'entitat HexagonVertex, amb id propi. Hi ha 19 × 6 = 114 files. Exemple: les sis files de l'hexàgon 10.

hexagon	vertex	posicio	Coordenades del vèrtex	On és
10	32	0	(0,866 ; −0,5)	dalt a la dreta
10	33	1	(0,866 ; 0,5)	baix a la dreta
10	21	2	(0 ; 1)	a baix
10	20	3	(−0,866 ; 0,5)	baix a l'esquerra
10	19	4	(−0,866 ; −0,5)	dalt a l'esquerra
10	31	5	(0 ; −1)	a dalt

Un mateix vèrtex té una posició diferent a cada hexàgon. El vèrtex 32 surt tres vegades: és la posició 0 de l'hexàgon 10, la 2 del 14 i la 4 del 15. Per saber quins hexàgons produeixen per a un poblat del vèrtex 32, es busquen les files amb `vertex = 32`.

```
hexagon,vertex,posicio
10,32,0
14,32,2
15,32,4
```

5. arestes.csv

Columna	Tipus	Significat
`codi`	int	Identificador de l'aresta dins dels CSV (1-72). Es guarda a Aresta.codi.
`vertexA`	int	Codi d'un extrem.
`vertexB`	int	Codi de l'altre extrem.

Cada costat d'hexàgon és una aresta; si el comparteixen dos hexàgons, només surt una vegada. L'aresta no té direcció: sempre `vertexA` < `vertexB`, però (19, 20) i (20, 19) serien la mateixa. Per això, quan es busca una aresta entre dos vèrtexs s'han de mirar les dues combinacions.

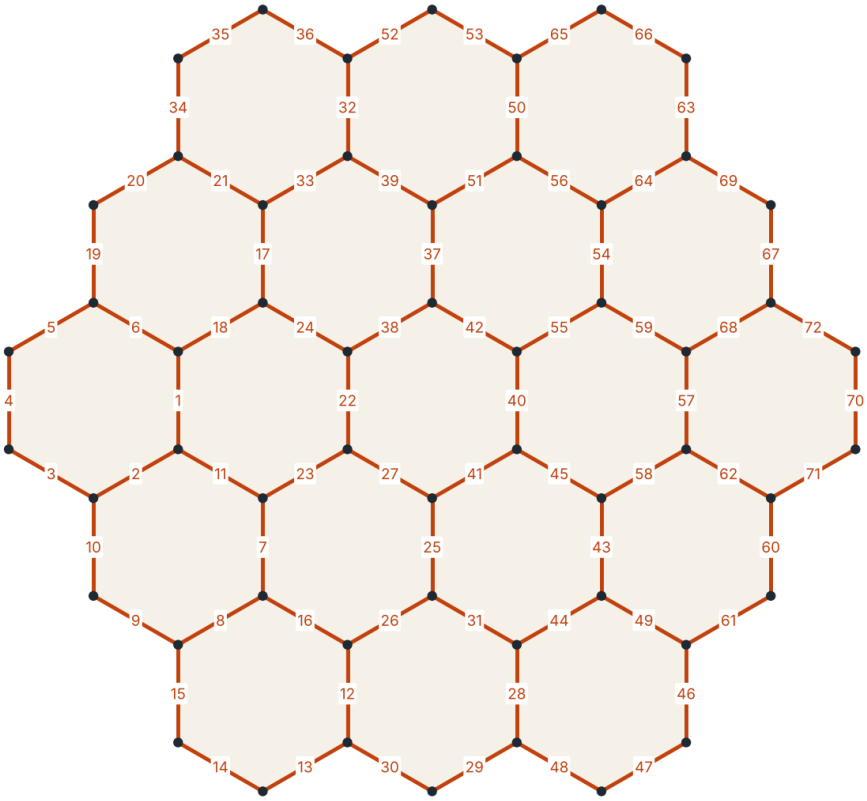


Figura 4. Codi de cada aresta.

Dos vèrtexs són **adjacents** si hi ha una aresta entre ells. Un vèrtex de l'interior té 3 arestes (36 vèrtexs) i un de la costa en té 2 o 3 (18 vèrtexs en tenen només 2). Exemple amb el vèrtex 32:

```
codi,vertexA,vertexB
40,32,33
42,31,32
55,32,43
→ veïns del vèrtex 32: 33, 31 i 43
```

6. Com es dibuixa el tauler

Per passar d'unitats a píxels només cal triar una mida (píxels per unitat) i on cau el centre del tauler a la pantalla. Amb una mida de 50 px el tauler fa 433 × 400 px, i cap en un panell de 500 × 450 amb el centre a (250, 225).

```
píxelX = OX + x · MIDA          píxelY = OY + y · MIDA

Exemple: vèrtex 32 = (0,866 ; −0,5), MIDA = 50, OX = 250, OY = 225
         → píxel (293, 200)
```

Dibuixar un hexàgon és unir els seus 6 vèrtexs **ordenats per posició**. Si no s'ordenen, el polígon surt creuat. Exemple amb Swing (els noms dels getters són orientatius):

```
static final double MIDA = 50, OX = 250, OY = 225;

int px(double x) { return (int) Math.round(OX + x * MIDA); }
int py(double y) { return (int) Math.round(OY + y * MIDA); }

Polygon poligon(Hexagon h) {
    Polygon p = new Polygon();
    h.getVertexs().stream()                        // List<HexagonVertex>
        .sorted(Comparator.comparingInt(HexagonVertex::getPosicio))
        .map(HexagonVertex::getVertex)
        .forEach(v -> p.addPoint(px(v.getX()), py(v.getY())));
    return p;
}
```

Amb el mateix sistema: una carretera és una línia de vertexA a vertexB; un poblat o una ciutat és un símbol a les coordenades del vèrtex; el número i el lladre van al centre de l'hexàgon (fórmula de l'apartat 2, o la mitjana dels seus 6 vèrtexs).

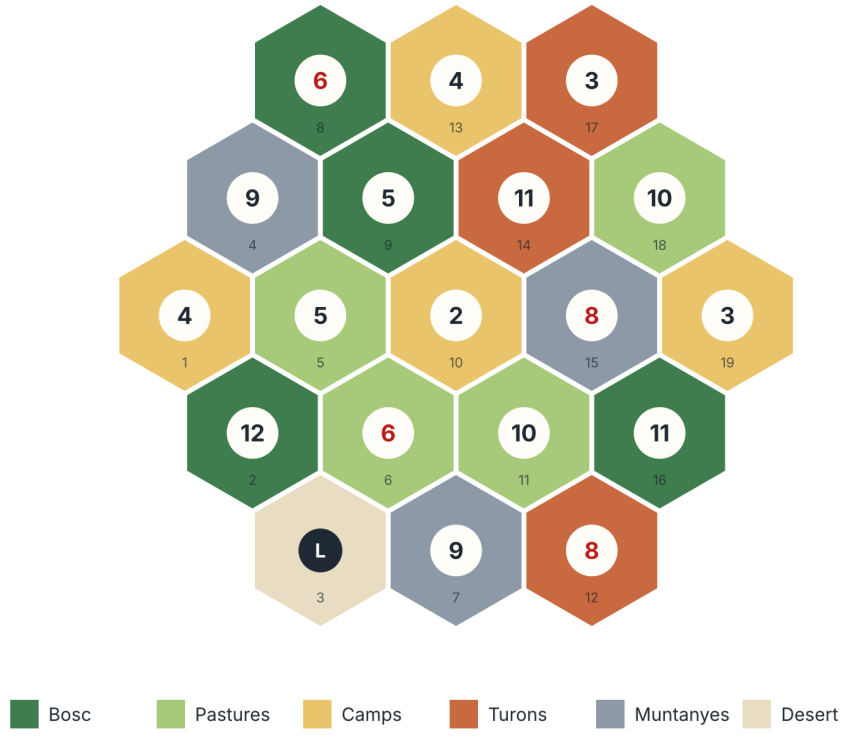


Figura 5. Exemple de partida dibuixada. Els terrenys i els números són un sorteig d'exemple: cada partida en tindrà uns altres. El número petit és el codi de l'hexàgon i la L marca el lladre, que comença al desert.

7. Què es pot saber a partir dels fitxers

Totes les regles del joc que depenen del tauler surten de les relacions anteriors:

Pregunta	Com es respon	Exemple
Quins vèrtexs té un hexàgon?	HexagonVertex amb aquest hexàgon	Hexàgon 10 → 32, 33, 21, 20, 19, 31
Quins hexàgons toca un vèrtex? (producció)	HexagonVertex amb aquest vèrtex	Vèrtex 32 → 10, 14, 15
Quins vèrtexs són veïns? (regla de la distància)	Arestes on el vèrtex és vertexA o vertexB; l'altre extrem és el veí	Vèrtex 32 → 33, 31, 43
Una carretera nova és connectada?	Algun dels seus dos vèrtexs té una construcció pròpia o hi arriba una altra carretera pròpia	Aresta 55 (32-43): cal tenir alguna cosa al 32 o al 43
Un poblat nou és connectat?	Alguna aresta del vèrtex té una carretera del mateix jugador	Vèrtex 32: arestes 40, 42 o 55

Comprovacions per a la càrrega

Després de carregar el tauler d'una partida, aquests comptes han de quadrar. Són útils per al document de proves:

- 19 hexàgons, 54 vèrtexs, 114 HexagonVertex i 72 arestes per partida.
- Cada hexàgon té exactament 6 vèrtexs, amb les posicions de 0 a 5 sense repetir.
- 18 vèrtexs toquen 1 hexàgon, 12 en toquen 2 i 24 en toquen 3.
- 18 vèrtexs tenen 2 arestes i 36 en tenen 3.

8. Les coordenades a Java

`Hexagon` ja té els atributs `q` i `r`. En canvi, al model actual `Vertex` només té `id` i `codi`, de manera que les columnes `x` i `y` del CSV es llegeixen però no es guarden. Si es vol poder dibuixar el tauler (per exemple, des de Spring o en una ampliació), n'hi ha prou d'afegir-les:

```
@Column(nullable = false)
private double x;

@Column(nullable = false)
private double y;

// a la càrrega
Vertex v = new Vertex(Integer.parseInt(fila[0]),
                      Double.parseDouble(fila[1]),
                      Double.parseDouble(fila[2]));
```

`Double.parseDouble` sempre fa servir el punt com a separador decimal, igual que el CSV, sigui quin sigui l'idioma de l'ordinador. El centre de l'hexàgon no cal guardar-lo: es calcula a partir de q i r o dels seus vèrtexs.